

Java Assignment 5– Inheritance

1. Simple Inheritance – Employee and Manager

Problem Statement

Create a Java program to demonstrate **Simple Inheritance** using `Employee` and `Manager` classes.

Superclass – `Employee`

Properties:

- `employeeId`
- `employeeName`
- `basicSalary`

Constructor:

Initialize all the `Employee` properties.

Methods:

- `calculateSalary()`
- `displayEmployeeDetails()`

Subclass – `Manager`

Additional Properties:

- `department`
- `bonus`

Constructor:

Initialize the `Employee` properties using `super()` and `Manager`-specific properties.

Methods:

- `calculateTotalSalary()`
- `displayManagerDetails()`

The program should display the employee ID, name, basic salary, department, bonus, and total salary.

2. Multilevel Inheritance – Vehicle, Car and ElectricCar

Problem Statement

Create a Java program to demonstrate **Multilevel Inheritance** using `Vehicle`, `Car`, and `ElectricCar`.

Superclass – `Vehicle`

Properties:

- `vehicleNo`
- `brand`
- `price`

Constructor:

Initialize all `Vehicle` properties.

Methods:

- `displayVehicleDetails()`
- `calculateTax()`

Subclass – `Car`

Inherit from `Vehicle`.

Additional Properties:

- `model`
- `fuelType`

Constructor:

Initialize `Vehicle` properties using `super()` and `Car` properties.

Methods:

- `displayCarDetails()`
- `calculateInsurance()`

Subclass – `ElectricCar`

Inherit from `Car`.

Additional Properties:

- `batteryCapacity`

- `chargingTime`

Constructor:

Initialize all inherited and `ElectricCar`-specific properties.

Methods:

- `calculateRange()`
- `displayElectricCarDetails()`

Create an `ElectricCar` object and display all vehicle, car, and electric-car details.

3. Hierarchical Inheritance – Bank Account

Problem Statement

Create a Java program to demonstrate **Hierarchical Inheritance** using `BankAccount` as the superclass and `SavingsAccount` and `CurrentAccount` as subclasses.

Superclass – BankAccount**Properties:**

- `accountNo`
- `accountHolderName`
- `balance`

Constructor:

Initialize all `BankAccount` properties.

Methods:

- `deposit()`
- `withdraw()`
- `displayAccountDetails()`

Subclass – SavingsAccount**Additional Property:**

- `interestRate`

Constructor:

Initialize `BankAccount` properties using `super()` and the interest rate.

Methods:

- `calculateInterest()`
- `displaySavingsDetails()`

Subclass – `CurrentAccount`

Additional Property:

- `overdraftLimit`

Constructor:

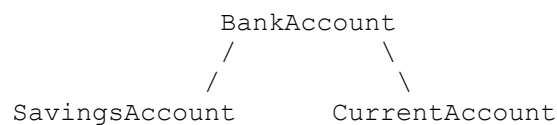
Initialize `BankAccount` properties using `super()` and the overdraft limit.

Methods:

- `checkOverdraftLimit()`
- `displayCurrentAccountDetails()`

Create objects for both `SavingsAccount` and `CurrentAccount` and display their respective details.

Inheritance



4. Hierarchical Inheritance – Product and Specialized Products

Problem Statement

Create a Java program to demonstrate **Hierarchical Inheritance** using `Product` as the superclass and `Electronics` and `Clothing` as subclasses.

Superclass – `Product`

Properties:

- `productId`
- `productName`
- `price`

Constructor:

Initialize all `Product` properties.

Methods:

- `calculateDiscount()`
- `displayProductDetails()`

Subclass – Electronics

Additional Properties:

- `brand`
- `warranty`

Constructor:

Initialize Product properties using `super()` and Electronics properties.

Methods:

- `calculateFinalPrice()`
- `displayElectronicsDetails()`

Subclass – Clothing

Additional Properties:

- `size`
- `material`

Constructor:

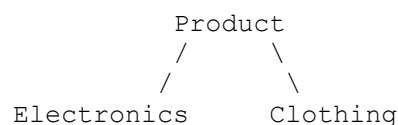
Initialize Product properties using `super()` and Clothing properties.

Methods:

- `calculateFinalPrice()`
- `displayClothingDetails()`

Create objects for both Electronics and Clothing and display their complete details.

Inheritance



5. Hierarchical Inheritance – Hospital Management

Problem Statement

Create a Java program to demonstrate **Hierarchical Inheritance** using `Person` as the superclass and `Doctor` and `Patient` as subclasses.

Superclass – `Person`

Properties:

- `personId`
- `personName`
- `age`

Constructor:

Initialize all `Person` properties.

Methods:

- `displayPersonDetails()`
- `checkAge()`

Subclass – `Doctor`

Additional Properties:

- `specialization`
- `consultationFee`

Constructor:

Initialize `Person` properties using `super()` and `Doctor`-specific properties.

Methods:

- `calculateConsultationAmount()`
- `displayDoctorDetails()`

Subclass – `Patient`

Additional Properties:

- `disease`
- `roomNumber`

Constructor:

Initialize `Person` properties using `super()` and `Patient`-specific properties.

Methods:

- `calculateRoomCharge()`
- `displayPatientDetails()`

Create objects for both `Doctor` and `Patient` and display their complete details.

Inheritance

